

Harness Architecture Standard

OriginID: OOF-OID-GE-HAS-2026-05-09-0001
Category: Governance & Enforcement
Subcategory: Execution Containment & Controlled Action Environments
Type: Derived Execution Containment Standard
Standard Role: Derived Standard under **Execution Layer — Enforcement Standard**
Version: 1.0
Status: Canonical · Open Standard
Effective Date: 9 May 2026
Parent Standard: **Execution Layer — Enforcement Standard**
Compatibility: OOF® Methodology OS™ · **Execution Layer — Enforcement Standard** · RIS · INTEGROS® · OGL™ · ART · OIB™ · EVIP®
AI-Readable: Yes
Authority: OOF®
Protection: MIP® — Methodological Intellectual Property
Canonical Language: English (UCL™)

Canonical Definition System

Harness Architecture Standard defines the structural conditions under which execution is contained, isolated, bounded, permission-governed, tool-restricted, and traceably released within a controlled operational environment.

Harness Architecture does not define whether execution is valid in principle.

Harness Architecture defines where and how valid execution is allowed to occur.

As a **derived standard under Execution Layer — Enforcement Standard**, Harness Architecture governs the **contained execution** environment through which actions, tool use, escalation paths, runtime access, and output release remain structurally controlled.

A. Standard Abstract

Modern autonomous systems no longer operate only through isolated internal logic.

They increasingly:

- invoke external tools
- trigger runtime actions
- access bounded environments
- produce outputs with operational consequences
- move from internal reasoning into external execution

- cross from decision into action

This creates a distinct governance problem.

Execution may be authorized and still remain unsafe if the environment of execution is not properly contained.

Harness Architecture Standard establishes the controlled execution environment required to ensure that actions occur only within bounded, restricted, traceable, and operationally governed containment structures.

B. Core Principle

A system cannot be considered execution-safe merely because an action is authorized.

Execution must also occur within a governed containment architecture.

C. Scope

Harness Architecture Standard applies to:

- AI agents
- tool-using systems
- autonomous execution runtimes
- enterprise automation environments
- code execution environments
- bounded runtime containers
- action-triggering orchestration systems
- controlled external interface systems
- systems that move from internal decision to external action

The standard applies wherever execution must remain contained, isolated, restricted, and traceable before real operational effect occurs.

D. System Position

Harness Architecture Standard operates as:

- a derived standard under **Execution Layer**
- an execution containment architecture layer
- a controlled action environment standard
- a boundary between internal decision and external execution
- a release-governance condition for operational outputs

It does not replace enforcement logic.

It provides the architecture through which enforcement remains

operationally contained.

E. Structural Base

Harness Architecture Standard is based on:

- bounded execution environments
- controlled tool access
- runtime isolation
- governed permission escalation
- traceable execution transition
- controlled output release
- containment before operational effect

A system must contain execution before external action may be treated as structurally governed.

F. Core Conditions

A Harness Architecture-compatible system must define:

1. Execution Boundary

The system must define where execution begins, where it is contained, and what remains outside the permitted action environment.

2. Tool Access Restriction

The system must define which tools, interfaces, or execution capabilities are available and under which conditions they may be invoked.

3. Runtime Isolation

The system must define how execution is isolated from unrelated runtimes, contexts, sessions, or uncontrolled operational pathways.

4. Permission Escalation Control

The system must define when execution may request higher authority, broader access, or expanded action scope, and how such escalation remains governed.

5. Traceable Execution Transition

The system must preserve traceability across the transition from internal decision to actual execution.

6. Controlled Output Release

The system must define what may leave the contained environment, under what conditions, and through which release boundaries.

7. Containment Integrity

No execution may be treated as valid if the containment environment can be silently bypassed, widened, or operationally escaped without governed restriction.

Non-bypassable logic applies at validated state only.

G. Methodology

Implementation of **Harness Architecture Standard** requires:

- 1. define the **contained execution** environment
- 2. define available tools and action surfaces
- 3. define runtime isolation boundaries
- 4. define permission escalation logic
- 5. define execution-to-output transition path
- 6. define release conditions for external effect
- 7. verify that execution remains bounded before operational consequence occurs

A system may be relied upon only after execution containment conditions are established.

H. Invalid Conditions

A system is considered Harness Architecture-invalid if:

- execution occurs outside defined containment boundaries
- tool access is broader than governed scope
- runtime isolation is missing or bypassable
- permission escalation is undefined or silently expanded
- execution cannot be traced across decision-to-action transition
- outputs leave the environment without governed release conditions
- containment exists only formally but not operationally

Authorized execution without containment does not satisfy this standard.

I. System Impact

Harness Architecture Standard transforms execution from:

- authorized action → into **contained action**
 - tool invocation → into **bounded operational access**
 - runtime activity → into **isolated controlled execution**
-

- output generation → into **governed release transition**

It establishes the environment through which execution becomes safely actionable rather than merely permissible.

J. Derived Standard Function

As a derived standard under **Execution Layer, Harness**

Architecture Standard defines the contained architecture through which execution governance becomes operationally enforceable.

It is designed to support derived modules such as:

- execution boundary modules
- tool access control modules
- runtime isolation modules
- permission escalation modules
- output release control modules

These do not replace the derived standard.

They extend its application.

K. Cross-Layer Dependency

Harness Architecture Standard operates in connection with:

- **Execution Layer — Enforcement Standard** for authorization, traceability, constraints, integrity, and outcome binding
- **RIS** for runtime integrity conditions
- **INTEGROS®** for non-bypassable boundaries
- **OGL™** for orchestration-aware execution control
- **ART** for real-time audit linkage
- **OIB™** for information boundary restrictions
- **EVIP®** for ethical execution limits

Execution containment becomes operationally meaningful only when these layers remain structurally aligned where applicable.

L. Structural Principle

Execution does not become controlled because it is permitted.

It becomes controlled only when the environment in which it occurs is itself governed.

Canonical Closing Statement

Module Architecture

[→ About Harness Architecture Standard](#)

[→ Module 1 — EBM — Execution Boundary Module](#)

[→ Module 2 — TACM — Tool Access Control Module](#)

[→ Module 3 — RIM — Runtime Isolation Module](#)

[→ Module 4 — PEM — Permission Escalation Module](#)

[→ Module 5 — ETTM — Execution Traceability Transition Module](#)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>